Bioinf Pipelines. HW2

Deadline: 22.04
Performed by Victor Florin

You have to implement a pipeline that:

1. Takes local fastq reads or downloads them from NCBI by ID (depending on the input params)
2. Runs QC on the raw reads
3. Trims the reads
4. Runs QC on the trimmed reads
5. Assembles them into a de novo reference or takes an input reference file (depending on the input params)
6. Maps the trimmed reads to the reference
7. Plots coverage

15 points if steps 1-4 are completed; 30 points if all steps are completed.
Advice: do not use large files for testing, consider E.Coli or some small virus. For example, you can use this HIV reads: DRR030302.

1. Preparing the environment and writing scripts

I chose SRR1175163.

- Organism: ***Dengue virus***
- Size: 9.4Mb

Library:

- Name: M105
- Instrument: Illumina MiSeq
- Strategy: WGS
- Source: GENOMIC
- Selection: RANDOM PCR
- Layout: **PAIRED**

Creating & activating the HW2 env:

```
conda create -n hw2 -y
```

```
conda activate hw2
```

Adding channels:

```
conda config --add channels bioconda
conda config --add channels conda-forge
```

Installing needed tools:

```
conda install -y fastqc trimmomatic bwa samtools sra-tools spades
```

Just in case:

```
fastqc --version
trimmomatic -version
bwa 2>&1 | grep "Version"
samtools --version
spades.py --version
```

```
(hw2) student@LAPTOP-NKRTACMG:~$ fastqc --version
FastQC v0.12.1
(hw2) student@LAPTOP-NKRTACMG:~$ trimmomatic -version
0.40
(hw2) student@LAPTOP-NKRTACMG:~$ bwa 2>&1 | grep "Version"
Version: 0.7.19-r1273
(hw2) student@LAPTOP-NKRTACMG:~$ ^C
(hw2) student@LAPTOP-NKRTACMG:~$ samtools --version
samtools 1.23.1
Using htslib 1.23.1
(hw2) student@LAPTOP-NKRTACMG:~$ spades.py --version
SPAdes genome assembler v4.2.0
```

Making structure - creating the project folder and filling it:

```
mkdir hw2_pipeline
cd hw2_pipeline
touch main.nf
touch nextflow.config
mkdir data
```

For nextflow.config:

```
cat > nextflow.config << 'EOF'
params {
    reads = null
```

```
    sra_id = null
    reference = null
}

process {
    executor = 'local'
}
EOF

# cat nextflow.config
```

```
(hw2) student@LAPTOP-NKRTACMG:~/hw2_pipeline$ cat nextflow.config
params {
    reads = null
    sra_id = null
    reference = null
}

process {
    executor = 'local'
}
(hw2) student@LAPTOP-NKRTACMG:~/hw2_pipeline$
```

For main.nf:

```
cat > main.nf << 'EOF'
params.reads = null
params.sra_id = null
params.reference = null
params.outdir = "results"

// =======================
// PROCESS: FETCH READS
// =======================
process fetch_reads {
    tag "$sra_id"
    publishDir "${params.outdir}/raw_reads", mode: 'copy'

    input:
        val sra_id

    output:
        tuple val(sra_id), path("${sra_id}_1.fastq"), path("${sra_id}_2.fastq")

    script:
    """
    fasterq-dump --split-files $sra_id
    """
```

```
}

// ========================
// PROCESS: FASTQC
// ========================
process fastqc {
    tag "$sample_id"
    publishDir "${params.outdir}/qc_${reads_type}", mode: 'copy'

    input:
        val reads_type
        tuple val(sample_id), path(r1), path(r2)

    output:
        path "*_fastqc.{zip,html}"

    script:
    """
    fastqc -t ${task.cpus} $r1 $r2
    """
}

// ========================
// PROCESS: TRIMMING
// ========================
process trimm {
    tag "$sample_id"
    publishDir "${params.outdir}/trimmed_reads", mode: 'copy', pattern: "*_p.fastq"

    input:
        tuple val(sample_id), path(r1), path(r2)

    output:
        tuple val(sample_id), path("${sample_id}_R1_p.fastq"),
path("${sample_id}_R2_p.fastq")

    script:
    """
    trimmomatic PE -threads ${task.cpus} \
        $r1 $r2 \
        ${sample_id}_R1_p.fastq ${sample_id}_R1_u.fastq \
        ${sample_id}_R2_p.fastq ${sample_id}_R2_u.fastq \
        LEADING:3 TRAILING:3 MINLEN:36
    """
}
```

```
// ========================
// WORKFLOW: TRIMMED QC
// ========================
workflow trimmed_qc_wf {
    take:
        trimmed_reads
    main:
        fastqc('trimmed', trimmed_reads)
    emit:
        fastqc.out
}

// ========================
// PROCESS: ASSEMBLY
// ========================
process assembly {
    tag "$sample_id"
    publishDir "${params.outdir}/assembly", mode: 'copy'

    input:
        tuple val(sample_id), path(r1), path(r2)

    output:
        tuple val(sample_id), path("${sample_id}_contigs.fasta")

    script:
    """
    spades.py -t ${task.cpus} -1 $r1 -2 $r2 -o spades_out
    cp spades_out/contigs.fasta ${sample_id}_contigs.fasta
    """
}

// ========================
// PROCESS: MAPPING
// ========================
process mapping {
    tag "$sample_id"
    publishDir "${params.outdir}/mapped", mode: 'copy'

    input:
        tuple val(sample_id), path(r1), path(r2), path(ref)

    output:
        tuple val(sample_id), path("${sample_id}.sorted.bam")

    script:
```

```
    """
    bwa index $ref
    bwa mem -t ${task.cpus} $ref $r1 $r2 | \
        samtools view -@ ${task.cpus} -bS - | \
        samtools sort -@ ${task.cpus} -o ${sample_id}.sorted.bam
    """
}

// =======================
// PROCESS: COVERAGE PLOT
// =======================
process plot_coverage {
    tag "$sample_id"
    publishDir "${params.outdir}/coverage", mode: 'copy'

    input:
        tuple val(sample_id), path(bam)
        path r_script  // принимаем R скрипт как входной файл

    output:
        tuple val(sample_id), path("${sample_id}_coverage.png"),
path("${sample_id}_depth.txt")

    script:
    """
    samtools depth $bam > ${sample_id}_depth.txt
    Rscript $r_script ${sample_id}
    """
}

// =======================
// MAIN WORKFLOW
// =======================
workflow {
    if (params.sra_id) {
        raw_reads_ch = fetch_reads(params.sra_id)
    } else if (params.reads) {
        raw_reads_ch = Channel.fromFilePairs(params.reads, checkIfExists: true)
    } else {
        error "Please provide either '--sra_id' or '--reads
\"data/*_{1,2}.fastq\"'"
    }

    fastqc('raw', raw_reads_ch)

    trimmed_reads_ch = trimm(raw_reads_ch)
```

```
    trimmed_qc_wf(trimmed_reads_ch)

    if (params.reference) {
        ref_ch = Channel.fromPath(params.reference, checkIfExists: true)

        mapping_in_ch = trimmed_reads_ch.combine(ref_ch)
    } else {
        assembly_ch = assembly(trimmed_reads_ch)
        mapping_in_ch = trimmed_reads_ch.join(assembly_ch)
    }

    bam_ch = mapping(mapping_in_ch)

    plot_coverage(bam_ch, file("plot_coverage.R"))
}
EOF

# cat main.nf | head -20 ; tail -20 main.nf ; wc -l main.nf
```

```
(hw2) student@LAPTOP-NKRTACMG:~/hw2_pipeline$ cat main.nf | head -20
params.reads = null
params.sra_id = null
params.reference = null
params.outdir = "results"

// ========================
// PROCESS: FETCH READS
// ========================
process fetch_reads {
    tag "$sra_id"
    publishDir "${params.outdir}/raw_reads", mode: 'copy'

    input:
        val sra_id

    output:
        tuple val(sra_id), path("${sra_id}_1.fastq"), path("${sra_id}_2.fastq")

    script:
    """
```

Running the pipeline on **SRR1175163**:

```
nextflow run main.nf --sra_id SRR1175163
# OR nextflow run main.nf --sra_id SRR1175163 -resume
```

```
(hw2) student@LAPTOP-NKRTACMG:~/hw2_pipeline$ nextflow run main.nf --sra_id SRR1175163 -resume

N E X T F L O W  ~  version 25.10.4

Launching `main.nf` [lonely_mcnulty] DSL2 - revision: ada4773cde

executor >  local (1)
[0f/0db4d2] fetch_reads (SRR1175163)          | 1 of 1, cached: 1 ✔
[5c/0ed3b6] fastqc (SRR1175163)               | 1 of 1, cached: 1 ✔
[7e/a0ff87] trimm (SRR1175163)                | 1 of 1, cached: 1 ✔
[0a/28089d] trimmed_qc_wf:fastqc (SRR1175163) | 1 of 1, cached: 1 ✔
[12/50a162] assembly (SRR1175163)             | 1 of 1, cached: 1 ✔
[47/78ad54] mapping (SRR1175163)              | 1 of 1, cached: 1 ✔
[71/a7b9a4] plot_coverage (SRR1175163)        | 1 of 1 ✔
```

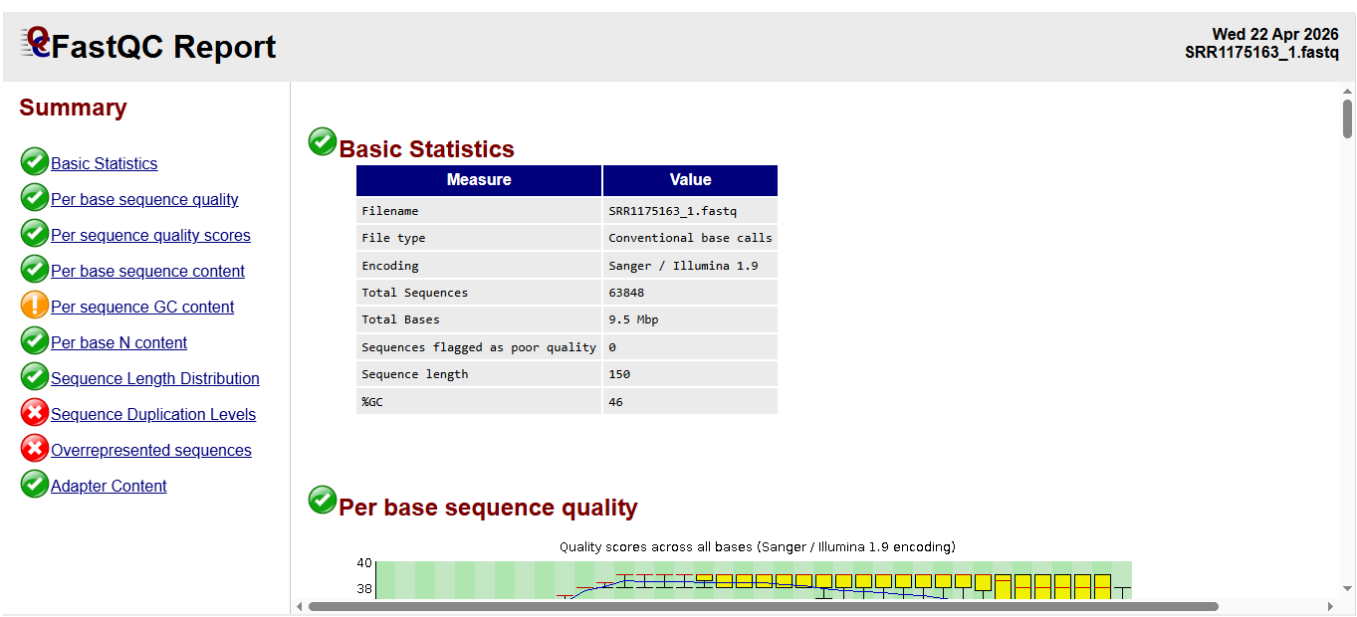
Cleaning temporary files if needed:

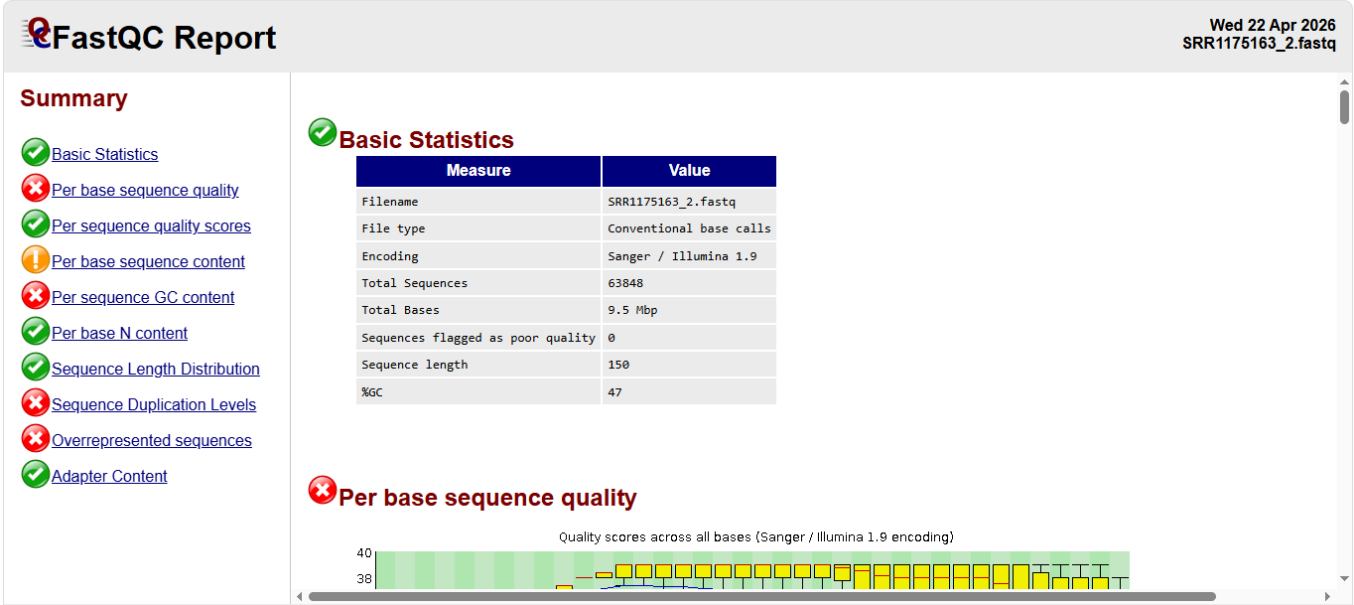
```
rm -rf .nextflow/ work/ results/
rm -f .nextflow.log*
# cp -r /home/student/hw2_pipeline
/mnt/c/Users/Student/Documents/GitHub/Bioinformatics-Pipelines/HW2/
```

---

2. QC on raw data

Done.





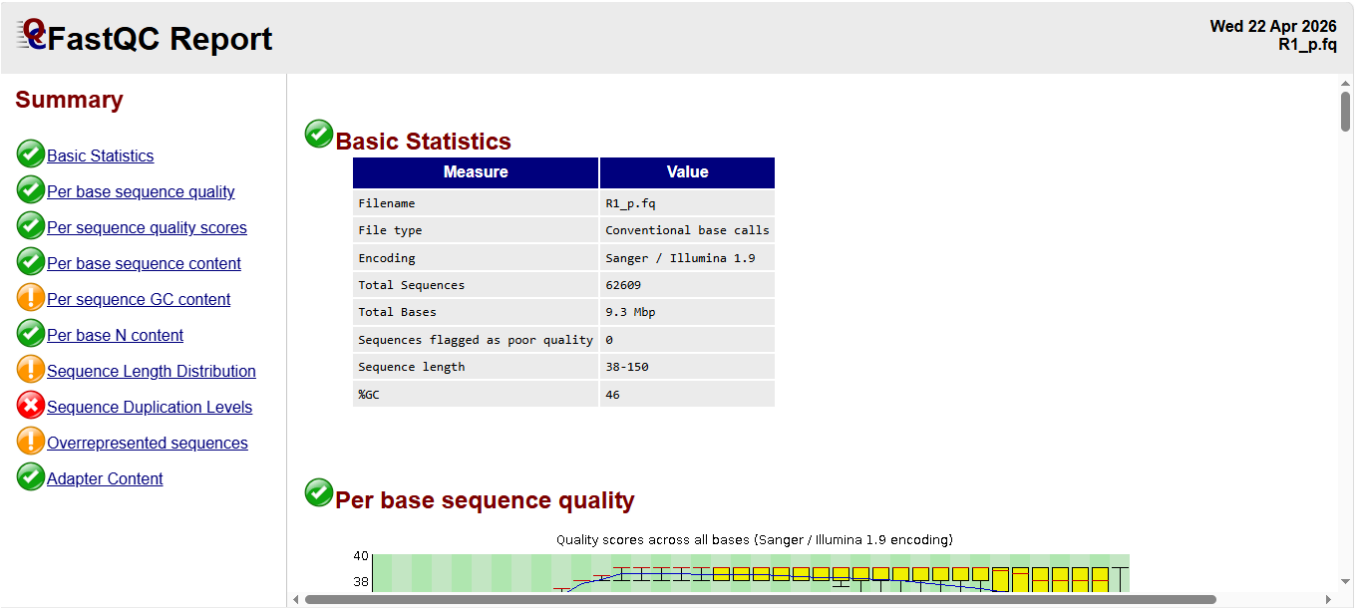
Measure	Value
Filename	SRR1175163_2.fastq
File type	Conventional base calls
Encoding	Sanger / Illumina 1.9
Total Sequences	63848
Total Bases	9.5 Mbp
Sequences flagged as poor quality	0
Sequence length	150
%GC	47

3. Trimming

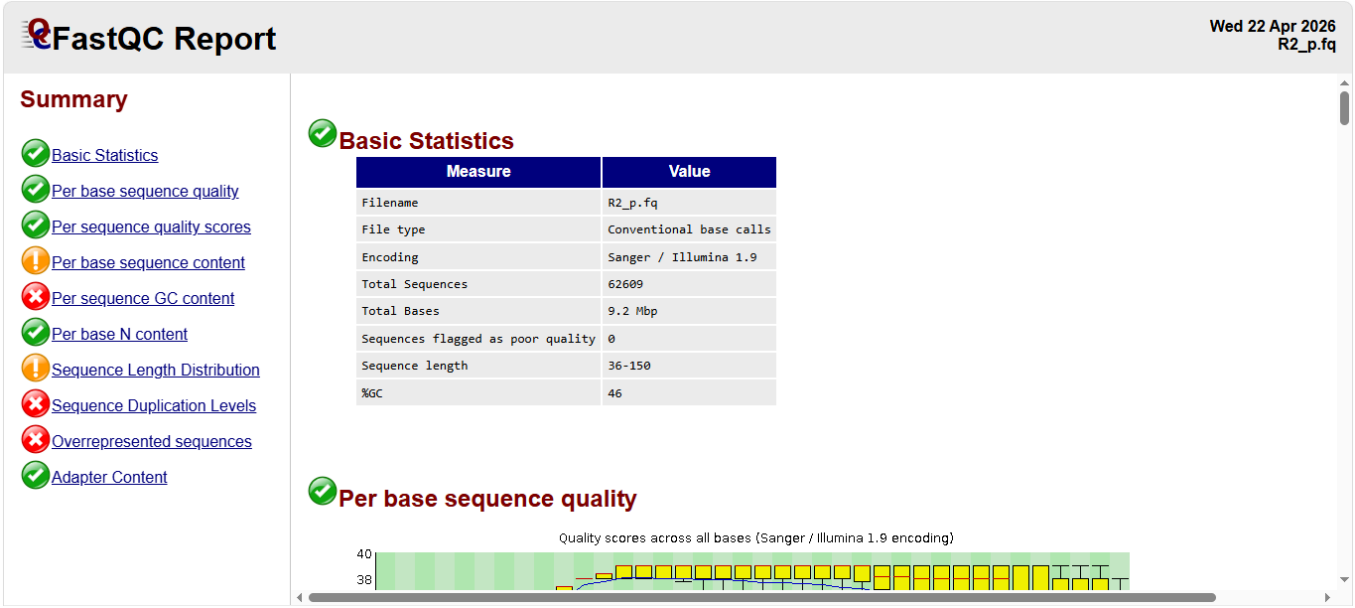
Done.

4. QC on trimmed data

Done.



Measure	Value
Filename	R1_p.fq
File type	Conventional base calls
Encoding	Sanger / Illumina 1.9
Total Sequences	62609
Total Bases	9.3 Mbp
Sequences flagged as poor quality	0
Sequence length	38-150
%GC	46



Measure	Value
Filename	R2_p.fq
File type	Conventional base calls
Encoding	Sanger / Illumina 1.9
Total Sequences	62609
Total Bases	9.2 Mbp
Sequences flagged as poor quality	0
Sequence length	36-150
%GC	46

---

5. Assembly

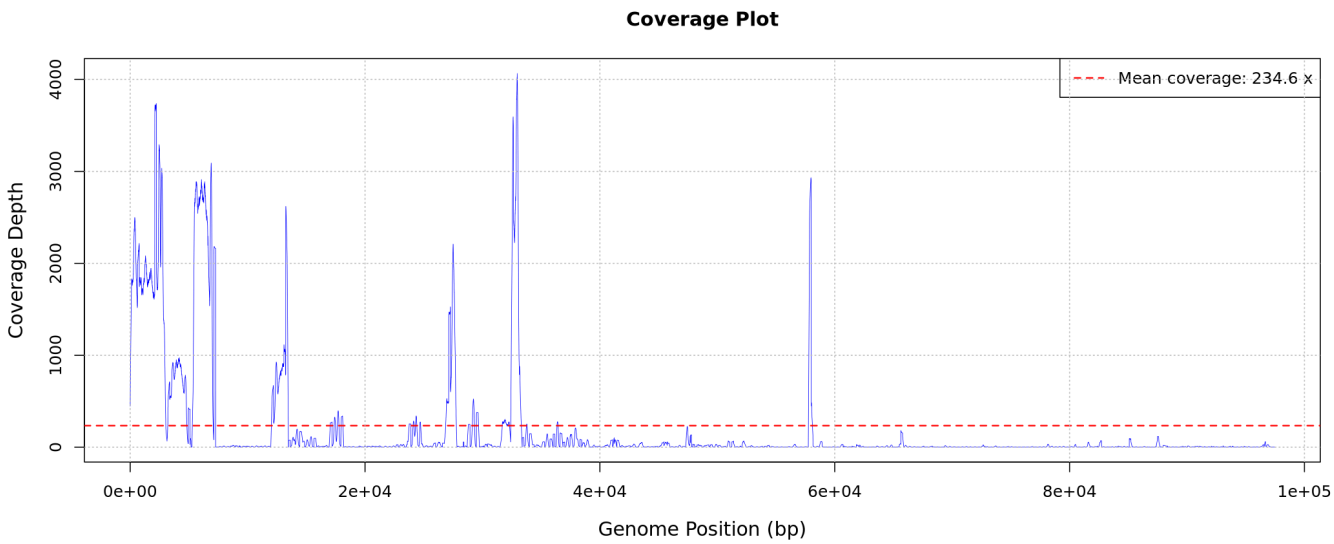
Done.

---

6. Mapping

Done.

---

7. Coverage plot

Let's look at coverage depth as a function of genomic position: